ALMATY TECHNOLOGICAL UNIVERSITY

Лабораторная работа № 7

По дисциплине

Проектирование и применение NoSQL-технологий

Тема: Создание графовой модели Neo4j и запросов Cypher

Группа: ИС 24-21

Студент: 3 курса

ФИО: Замякин Константин Дмитриевич

Преподаватель: Рашидинов.Д.Р

Цель работы

Сформировать практические навыки анализа предметной области, проектирования property graph-модели, создания узлов и направленных отношений в Neo4j и реализации запросов Cypher для поиска связанных данных и многошаговых обходов.

Индивидуальный вариант

Вариант 1 - «Университет». Узлы: Student, Course, Teacher. Отношения: ENROLLED_IN, TEACHES, REQUIRES. Уровень сложности: **повышенный** (ограниченный обход переменной длины, сравнение моделей, анализ масштабирования). Повышенное задание варианта: найти цепочку пререквизитов до 3 уровней.

Задачи работы

- проанализировать пользовательские вопросы предметной области «Университет» и составить таблицу паттернов доступа;
- обосновать выбор графовой модели и сравнить её с документной;
- спроектировать property graph: метки, свойства, типы и направления отношений, уникальные идентификаторы;
- создать граф не менее чем из 15 узлов и 20 отношений;
- реализовать не менее 7 запросов Cypher, включая двухшаговый обход, обход переменной длины с ограниченной глубиной и агрегирование count();
- изменить свойства узла и отношения, безопасно удалить тестовую связь;
- проанализировать преимущества, ограничения и сценарии роста графа.

Оборудование и программное обеспечение

- персональный компьютер;
- Docker Desktop, контейнер neo4j-lab7 (образ neo4j:latest, Neo4j 2026.09.0), порты 7474 (HTTP) и 7687 (Bolt);
- Neo4j Browser (http://localhost:7474), база данных neo4j, язык Cypher 25;
- искусственные учебные данные без персональных данных.

1. Теоретические сведения

1.1. Property graph

В Neo4j данные представляются графом свойств: **узлы** (nodes) описывают объекты, **отношения** (relationships) соединяют узлы, **свойства** (properties) хранят атрибуты узлов и отношений. **Метки** (labels) группируют узлы по ролям, например Student, Course или Teacher. Отношение всегда имеет тип и направление, например (:Student)-[:ENROLLED_IN]->(:Course).

1.2. Когда графовая модель оправдана

Графовая модель полезна, когда ключевые запросы зависят не только от атрибутов объектов, но и от цепочек связей: пререквизиты дисциплин, друзья друзей, маршруты, рекомендации, зависимости компонентов. Если основная задача - хранение автономных документов без интенсивных обходов, документная модель может быть проще.

1.3. Cypher

Cypher - декларативный язык запросов к property graph. Шаблон графа записывается визуально: узел - в круглых скобках, отношение - в квадратных, направление - стрелкой.

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course)
RETURN s.name, c.title;
```

1.4. CREATE и MERGE

CREATE всегда создаёт новый элемент. **MERGE** ищет заданный шаблон и создаёт его, только если совпадение отсутствует; он удобен для повторяемой загрузки, но требует уникальных идентификаторов. Поэтому в работе перед загрузкой данных созданы ограничения уникальности.

1.5. Обходы и ограничение глубины

Главное преимущество графовой модели проявляется в обходах: запрос может искать как непосредственную связь, так и путь длиной в несколько отношений. Глубину обхода переменной длины (например, *1..3) необходимо ограничивать, потому что на больших графах число путей растёт очень быстро.

1.6. Основные понятия

- **узел** - вершина графа, объект предметной области;
- **отношение** - направленная типизированная связь между двумя узлами;
- **свойство** - пара «ключ - значение» у узла или отношения;
- **label** - метка, определяющая роль узла;
- **путь (path)** - последовательность узлов и отношений;
- **ограничение уникальности** - гарантия того, что значение свойства не повторяется у узлов с данной меткой.

2. Ход выполнения лабораторной работы

2.1. Подготовка среды

Neo4j запущен в Docker Desktop как контейнер neo4j-lab7 (образ neo4j:latest, версия Neo4j 2026.09.0); статус контейнера - Running, опубликованы порты 7474 (Neo4j Browser) и 7687 (Bolt). В журнале контейнера видно, что сервер запущен (рисунок 1).

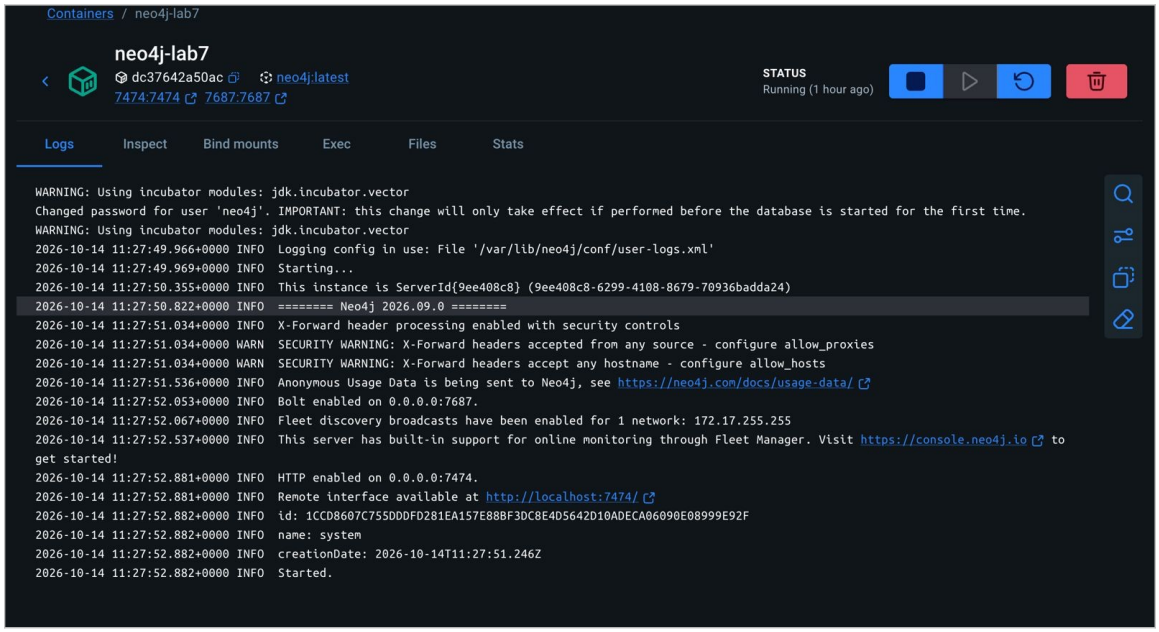


Рисунок 1 - Контейнер neo4j-lab7 в Docker Desktop: статус Running и журнал запуска

Работа выполнена в Neo4j Browser. Подключение выполняется к адресу localhost (протокол neo4j://, порт 7687), пользователь neo4j; пароль на скриншоте скрыт. Для проверки соединения выполнен запрос (рисунки 2 и 3):

```
RETURN 'Neo4j works' AS status;
```

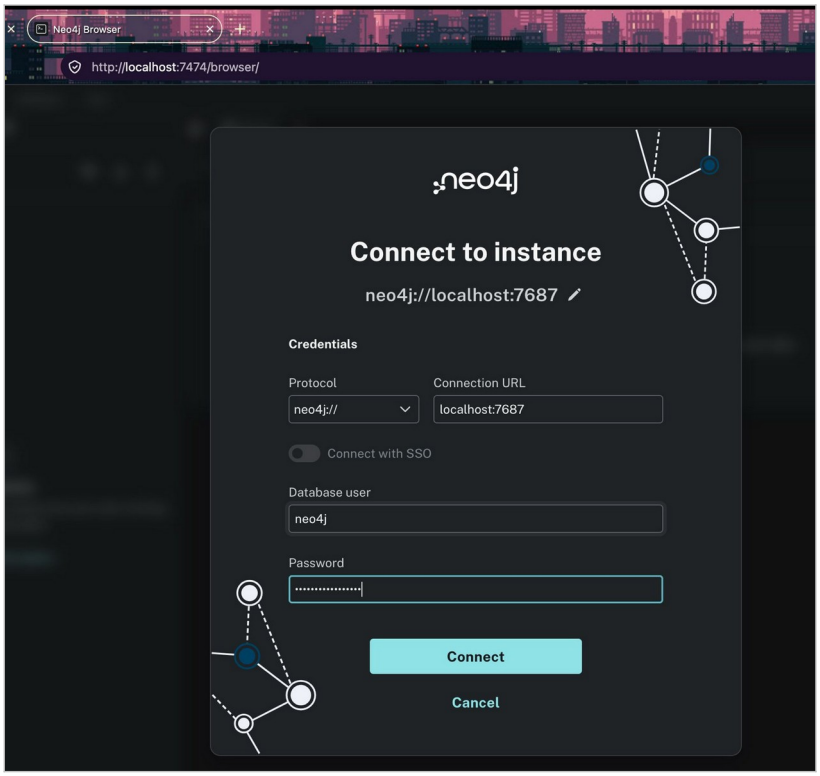


Рисунок 2 - Подключение к экземпляру Neo4j (neo4j://localhost:7687)

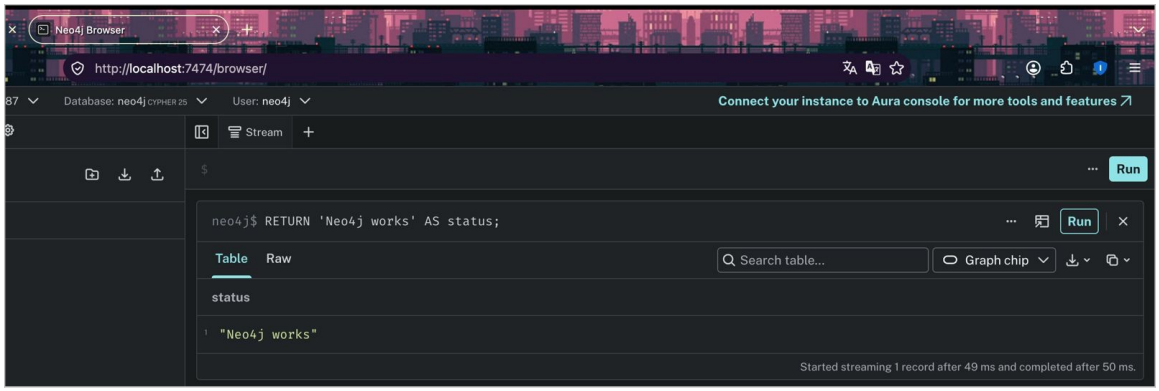


Рисунок 3 - Проверка соединения: запрос RETURN 'Neo4j works'

Использованы только искусственные учебные данные; реальные персональные данные в базу не загружались.

2.2. Анализ требований и паттерны доступа (этапы 1-2)

Роли пользователей: студент (смотрит свои дисциплины), преподаватель (смотрит студентов своих дисциплин), сотрудник учебной части (анализирует

загрузку дисциплин и проверяет пререквизиты). Паттерны доступа приведены в таблице 1.

Таблица 1 - Паттерны доступа

Паттерн доступа	**Частота**	**Способ реализации**
Студенты, записанные на дисциплину	высокая	один шаг по отношению ENROLLED_IN
Студенты старших курсов (year ≥ 3)	средняя	фильтрация по свойству узла, сортировка и LIMIT
Студенты и дисциплины конкретного преподавателя	средняя	двухшаговый обход Student - Course - Teacher
Цепочка пререквизитов дисциплины до 3 уровней	средняя	обход переменной длины REQUIRES*1..3
Число студентов на каждой дисциплине	высокая	агрегирование count() с группировкой по Course
Нагрузка преподавателя (дисциплины и студенты)	средняя	count(DISTINCT) по пути Teacher - Course - Student
Студенты дисциплины A, не записанные на дисциплину B	низкая	WHERE NOT с шаблоном отношения

Операции записи: зачисление студента на дисциплину (создание отношения ENROLLED_IN), изменение свойств узлов и отношений, отчисление с дисциплины (удаление отношения). Ключевые вопросы (какие дисциплины связаны, кто кого учит, что нужно пройти раньше) являются вопросами о связях, а не об отдельных атрибутах.

2.3. Обоснование модели и сравнение с документной (этап 2)

Для предметной области «Университет» существенны цепочки связей: студент - дисциплина - преподаватель и дисциплина - пререквизит - пререквизит. Графовая модель хранит эти связи как отношения, поэтому обход выполняется без соединения таблиц и без ручного поиска по спискам идентификаторов. Сравнение с документной моделью приведено в таблице 2.

Таблица 2 - Сравнение графовой и документной моделей для варианта «Университет»

Критерий	**Граф (Neo4j)**	**Документная модель**
Хранение связей	отношение - самостоятельный элемент с типом, направлением и свойствами	списки идентификаторов внутри документов (например, список курсов студента)
Цепочка пререквизитов	один запрос REQUIRES*1..3	несколько последовательных запросов в приложении или специальные операторы рекурсивного поиска
Зачисление и отчисление	создание или удаление одного отношения	правка списков в нескольких документах
Риск рассинхронизации	ниже: связь хранится один раз	выше: связь дублируется в двух документах
Карточка одного объекта	нужен запрос с обходом	проще: документ читается целиком
Когда проще	много обходов и связей	автономные документы без обходов

Вывод: для данного варианта графовая модель оправдана, так как главные паттерны доступа (пререквизиты, студенты преподавателя, связанные дисциплины) являются обходами. Документная модель была бы проще, если бы требовалось только хранить карточки студентов и читать их целиком.

2.4. Проектирование графа (этап 3)

Схема графа показана на рисунке 4. Описание узлов приведено в таблице 3, отношений - в таблице 4.

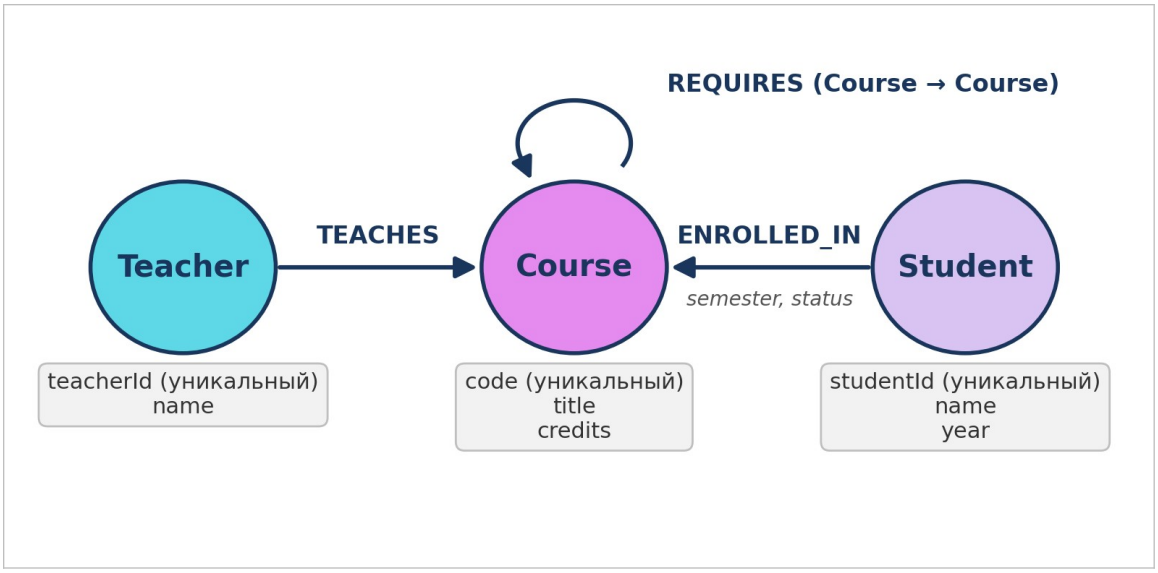


Рисунок 4 - Схема графовой модели «Университет»

Таблица 3 - Узлы графа

Метка	**Свойства**	**Уникальный ключ**	**Назначение**
Student	studentId, name, year	studentId	студент
Course	code, title, credits	code	дисциплина
Teacher	teacherId, name	teacherId	преподаватель

Таблица 4 - Отношения графа

Тип	**Направление**	**Свойства**	**Смысл**
ENROLLED_IN	(Student) → (Course)	semester, status	студент записан на дисциплину
TEACHES	(Teacher) → (Course)	нет	преподаватель ведёт дисциплину
REQUIRES	(Course) → (Course)	нет	дисциплина требует пререквизит

Примеры путей: (:Teacher)-[:TEACHES]->(:Course)<-[:ENROLLED_IN]-(:Student) связывает преподавателя со студентами; (ADV401)-[:REQUIRES]->(ALG202)-[:REQUIRES]->(PROG101) задаёт цепочку пререквизитов.

Направление выбрано по смыслу глагола: студент записывается на дисциплину, преподаватель ведёт дисциплину, дисциплина требует пререквизит. Дисциплина сделана узлом, а не свойством студента, потому что она участвует в нескольких видах связей. Семестр и статус записи - свойства отношения ENROLLED_IN, так как описывают сам факт зачисления. Прямая связь «студент - преподаватель» не хранится: она выводится через дисциплину, что исключает дублирование.

Чтобы MERGE не создавал дубли, а поиск по ключу был быстрым, созданы ограничения уникальности:

```
CREATE CONSTRAINT student_id IF NOT EXISTS
FOR (s:Student) REQUIRE s.studentId IS UNIQUE;
CREATE CONSTRAINT teacher_id IF NOT EXISTS
FOR (t:Teacher) REQUIRE t.teacherId IS UNIQUE;
CREATE CONSTRAINT course_code IF NOT EXISTS
FOR (c:Course) REQUIRE c.code IS UNIQUE;
SHOW CONSTRAINTS;
```

Команда SHOW CONSTRAINTS вернула три ограничения NODE_PROPERTY_UNIQUENESS (рисунок 5).

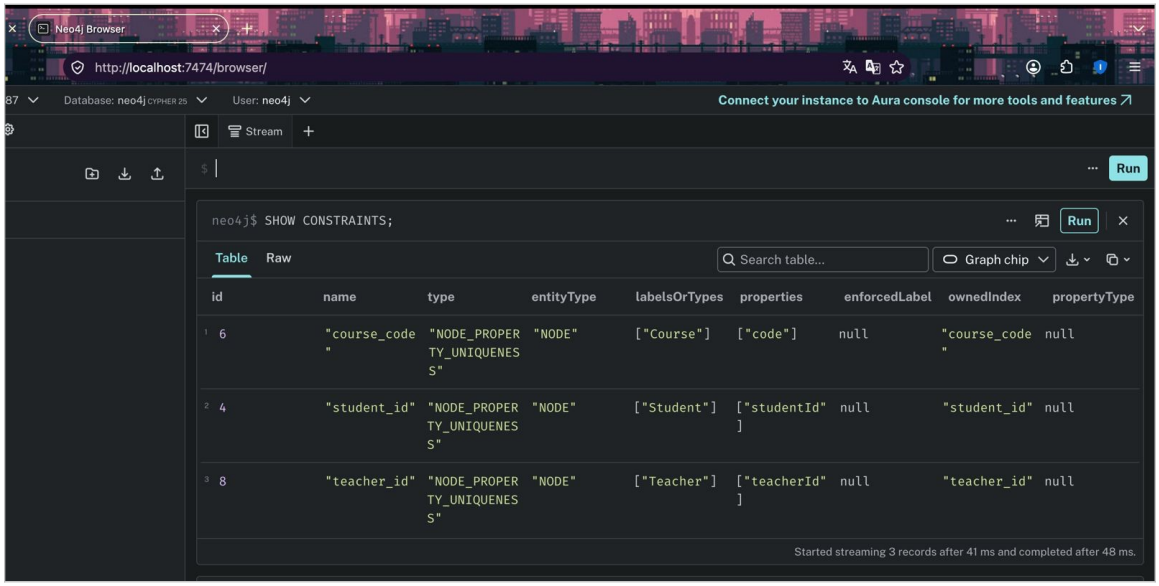


Рисунок 5 - Ограничения уникальности (SHOW CONSTRAINTS)

2.5. Создание графа (этап 4)

Данные загружены командами UNWIND + MERGE (повторный запуск не создаёт дублей). Сначала созданы узлы:

```
UNWIND [
  {id:1001, name:'Aruzhan', year:3}, {id:1002, name:'Dias', year:3},
  {id:1003, name:'Madina',  year:2}, {id:1004, name:'Zhanna', year:2},
  {id:1005, name:'Aigerim', year:3}, {id:1006, name:'Nurlan', year:4},
  {id:1007, name:'Timur',  year:2}, {id:1008, name:'Erlan', year:2}
] AS row
MERGE (s:Student {studentId: row.id})
SET s.name = row.name, s.year = row.year;

UNWIND [
  {id:501, name:'A. Teacher'}, {id:502, name:'B. Teacher'},
  {id:503, name:'C. Teacher'}
] AS row
MERGE (t:Teacher {teacherId: row.id})
SET t.name = row.name;

UNWIND [
  {code:'PROG101',  title:'Programming Basics',      credits:5},
  {code:'MATH101',  title:'Discrete Mathematics',    credits:4},
  {code:'DB201',    title:'Database Systems',        credits:5},
  {code:'ALG202',   title:'Algorithms',              credits:5},
  {code:'NOSQL301', title:'NoSQL Technologies',      credits:5},
  {code:'WEB210',   title:'Web Development',         credits:4},
  {code:'ADV401',   title:'Advanced Data Engineering', credits:5}
] AS row
MERGE (c:Course {code: row.code})
SET c.title = row.title, c.credits = row.credits;
```

Создано 8 узлов Student, 3 узла Teacher и 7 узлов Course - всего 18 узлов (рисунок 6).

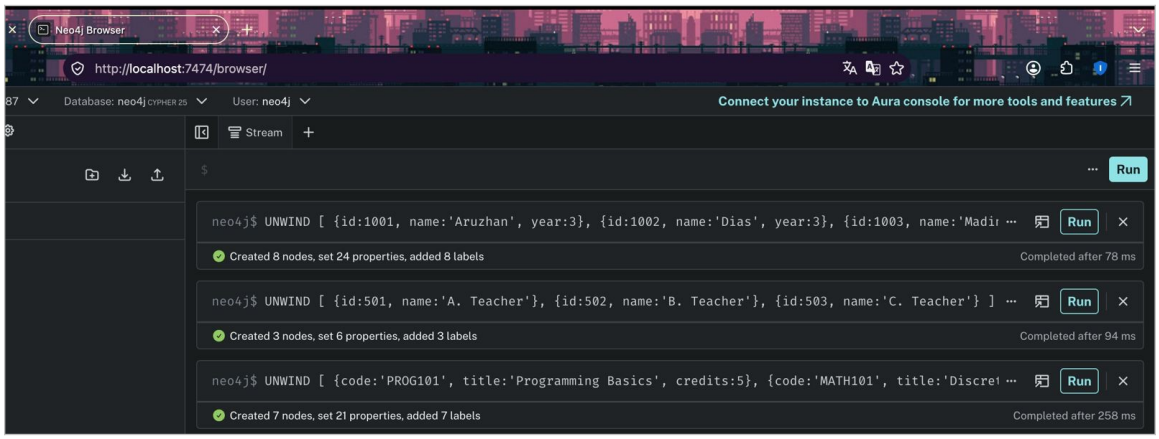


Рисунок 6 - Создание узлов Student, Teacher и Course (UNWIND + MERGE)

Затем созданы направленные отношения:

```
UNWIND [[1001,'NOSQL301'],[1001,'DB201'],[1002,'NOSQL301'],
  [1002,'WEB210'],[1003,'DB201'],[1003,'ALG202'],[1004,'PROG101'],
  [1004,'WEB210'],[1005,'NOSQL301'],[1005,'ALG202'],
  [1006,'NOSQL301'],[1006,'ADV401'],[1007,'PROG101'],
```

```
  [1007,'MATH101'],[1008,'DB201'],[1008,'PROG101']] AS pair
MATCH (s:Student {studentId: pair[0]}), (c:Course {code: pair[1]})
MERGE (s)-[:ENROLLED_IN {semester:'2026-Fall', status:'active'}]->(c);

UNWIND [[501,'PROG101'],[501,'ALG202'],[502,'DB201'],
  [502,'NOSQL301'],[502,'ADV401'],[503,'MATH101'],
  [503,'WEB210']] AS pair
MATCH (t:Teacher {teacherId: pair[0]}), (c:Course {code: pair[1]})
MERGE (t)-[:TEACHES]->(c);

UNWIND [['DB201','PROG101'],['ALG202','PROG101'],
  ['ALG202','MATH101'],['NOSQL301','DB201'],['WEB210','PROG101'],
  ['ADV401','ALG202'],['ADV401','NOSQL301']] AS pair
MATCH (a:Course {code: pair[0]}), (b:Course {code: pair[1]})
MERGE (a)-[:REQUIRES]->(b);
```

Создано 16 отношений ENROLLED_IN, 7 TEACHES и 7 REQUIRES - всего 30 отношений (рисунок 7). Neo4j вывел информационное уведомление 03N90 «Cartesian product»: в команде MATCH перечислены два независимых шаблона. Это не ошибка: каждый шаблон находит ровно один узел по уникальному ключу, поэтому результат корректен (создано ожидаемое число отношений).

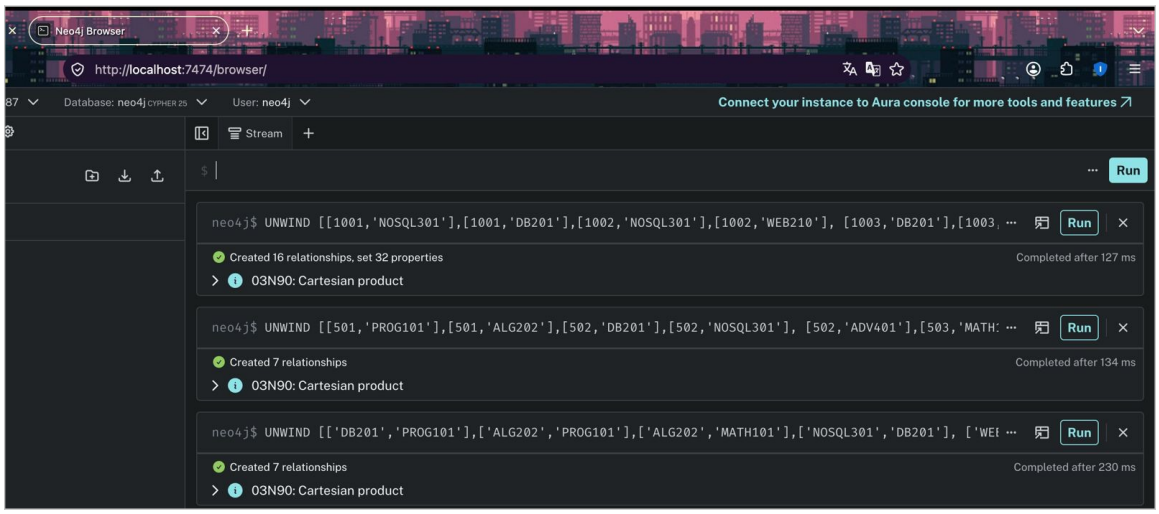


Рисунок 7 - Создание отношений ENROLLED_IN, TEACHES и REQUIRES

Исходные данные сведены в таблицы 5 и 6.

Таблица 5 - Дисциплины, преподаватели и пререквизиты

Код	Название	Преподаватель	Требует (REQUIRES)
PROG101	Programming Basics	A. Teacher	-
MATH101	Discrete Mathematics	C. Teacher	-
DB201	Database Systems	B. Teacher	PROG101
ALG202	Algorithms	A. Teacher	PROG101, MATH101
NOSQL301	NoSQL Technologies	B. Teacher	DB201
WEB210	Web Development	C. Teacher	PROG101
ADV401	Advanced Data Engineering	B. Teacher	ALG202, NOSQL301

Таблица 6 - Записи студентов на дисциплины (ENROLLED_IN, 2026-Fall)

Студент	Дисциплины
Aruzhan	NOSQL301, DB201
Dias	NOSQL301, WEB210
Madina	DB201, ALG202
Erlan	DB201, PROG101
Aigerim	NOSQL301, ALG202
Nurlan	NOSQL301, ADV401
Zhanna	PROG101, WEB210
Timur	PROG101, MATH101

Проверка структуры выполнена двумя запросами (рисунок 8):

```
MATCH (n) RETURN labels(n) AS labels, count(*) AS qty;
MATCH ()-[r]->() RETURN type(r) AS relation, count(*) AS qty;
```

Результат: Course - 7, Teacher - 3, Student - 8; REQUIRES - 7, TEACHES - 7, ENROLLED_IN - 16. Это совпадает с планом: 18 узлов и 30 отношений (требование - не менее 15 и 20).

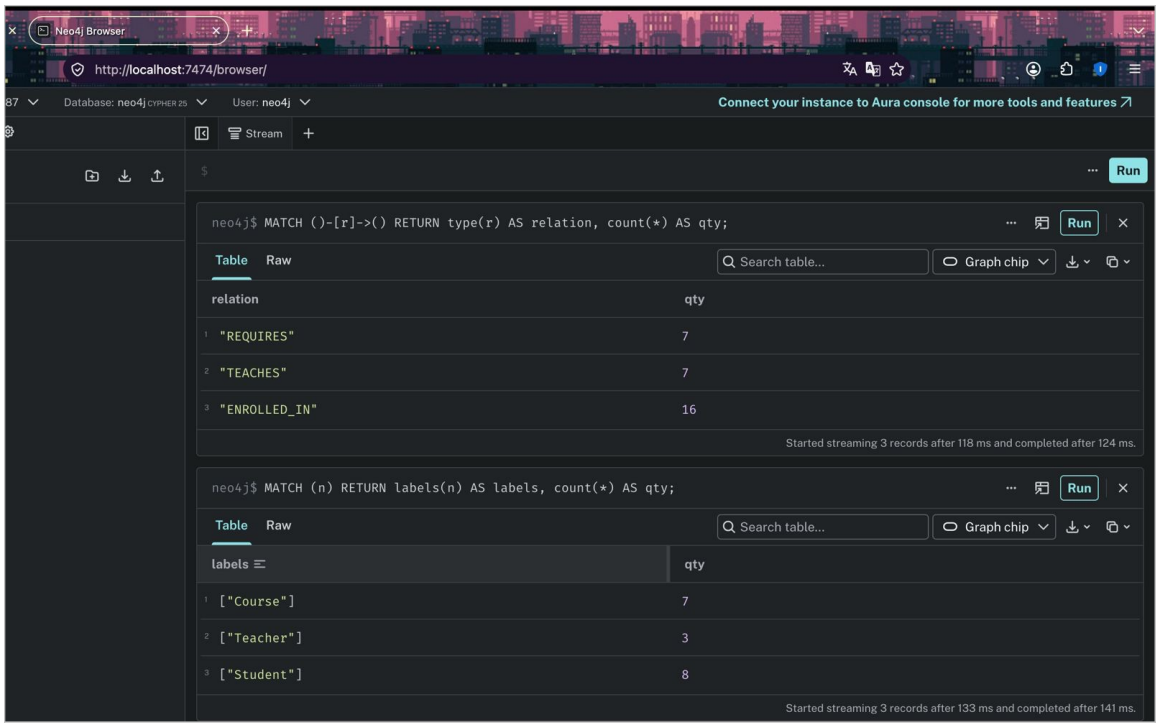


Рисунок 8 - Проверка структуры: число узлов по меткам и отношений по типам

Весь граф выведен запросом MATCH (n)-[r]->(m) RETURN n, r, m; (рисунок 9). Среди 18 узлов нет изолированных, а студенты, преподаватели и дисциплины связаны ожидаемыми направленными отношениями.

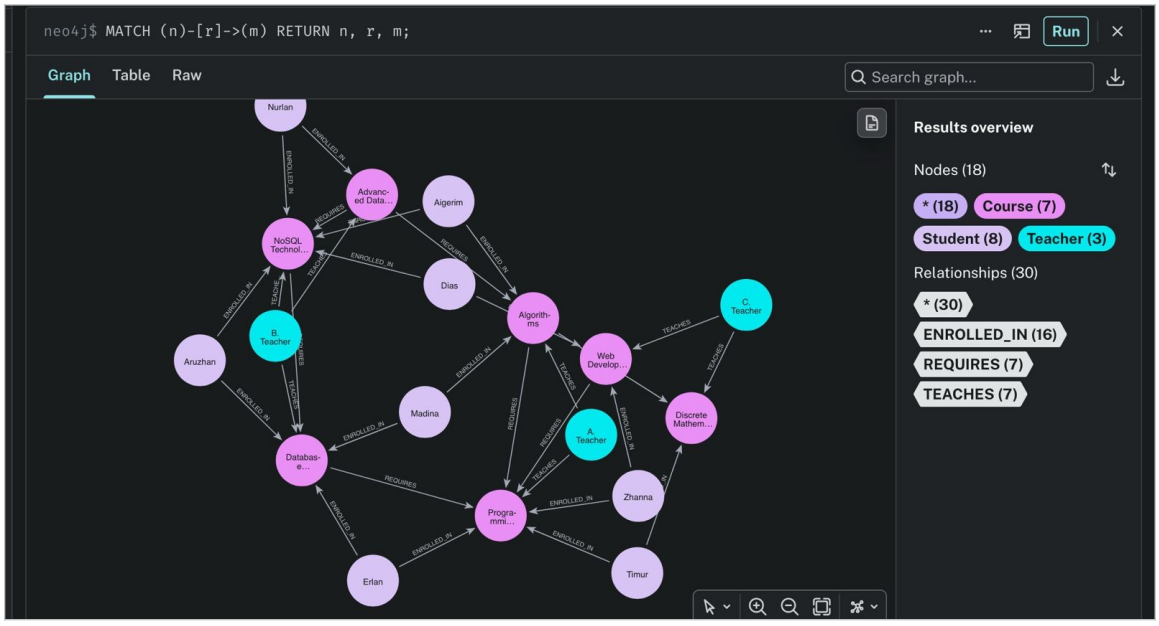


Рисунок 9 - Визуализация графа: 18 узлов и 30 отношений

2.6. Выполнение запросов (этап 5)

Все запросы строятся по связному шаблону (без перебора несвязанных пар), поэтому не порождают случайного декартова произведения.

Q1. Простой MATCH с фильтрацией, сортировкой и ограничением

```
MATCH (s:Student) WHERE s.year >= 3
RETURN s.name, s.year ORDER BY s.name LIMIT 5;
```

Запрос вернул 4 студентов 3 курса и старше: Aigerim, Aruzhan, Dias (3 курс) и Nurlan (4 курс). Лимит 5 не достигнут, остальные четыре студента учатся на более ранних курсах (рисунок 10).

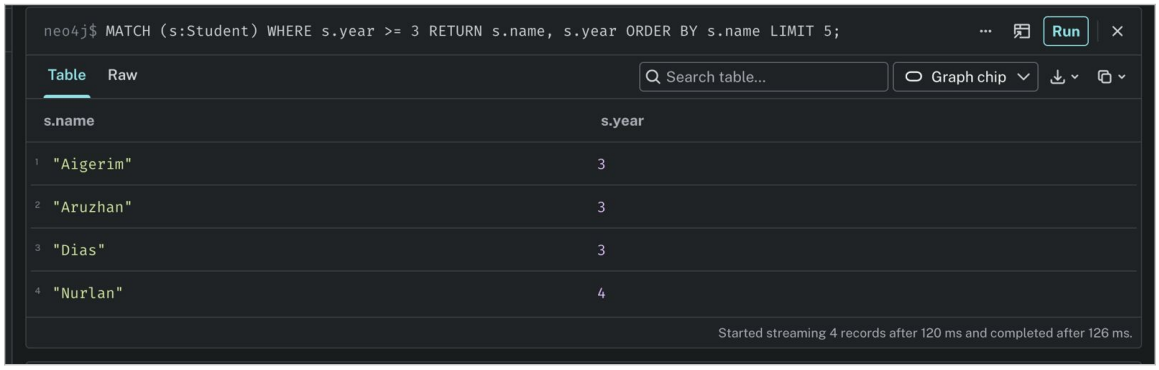


Рисунок 10 - Q1: студенты с year >= 3, сортировка по имени

Q2. Запрос по отношению: студенты дисциплины NOSQL301

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {code:'NOSQL301'})
RETURN s.studentId, s.name ORDER BY s.name;
```

Дисциплина NoSQL Technologies имеет 4 записанных студента: Aigerim (1005), Aruzhan (1001), Dias (1002) и Nurlan (1006) (рисунок 11).

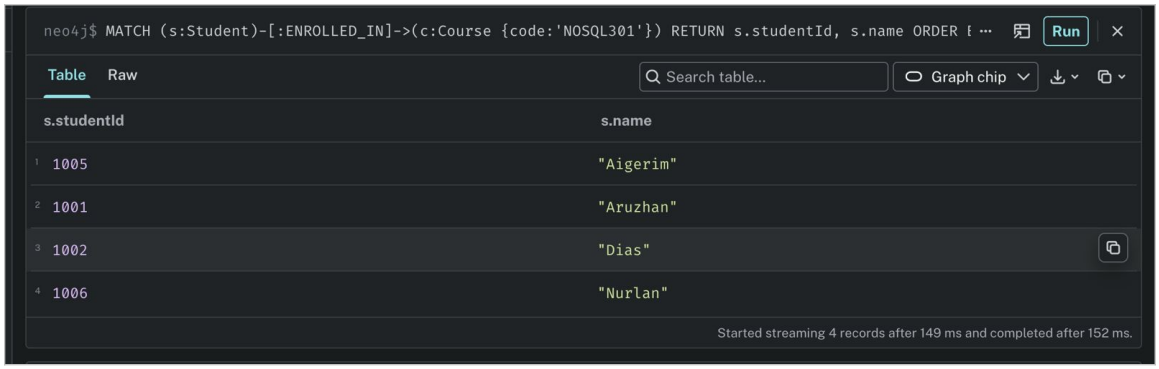


Рисунок 11 - Q2: студенты, записанные на NOSQL301

Q3. Двухшаговый обход: студенты и дисциплины преподавателя 502

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course)<-[:TEACHES]-
      (t:Teacher {teacherId:502})
RETURN DISTINCT s.name, c.title, t.name
ORDER BY c.title, s.name;
```

Обход проходит два отношения: от студента к дисциплине и от дисциплины к преподавателю B. Teacher. Получено 8 строк: Advanced Data Engineering - 1 студент, Database Systems - 3, NoSQL Technologies - 4 (рисунок 12). Aruzhan и Nurlan встречаются дважды, так как посещают по две дисциплины этого преподавателя, поэтому всего у B. Teacher 6 различных студентов (см. Q7).

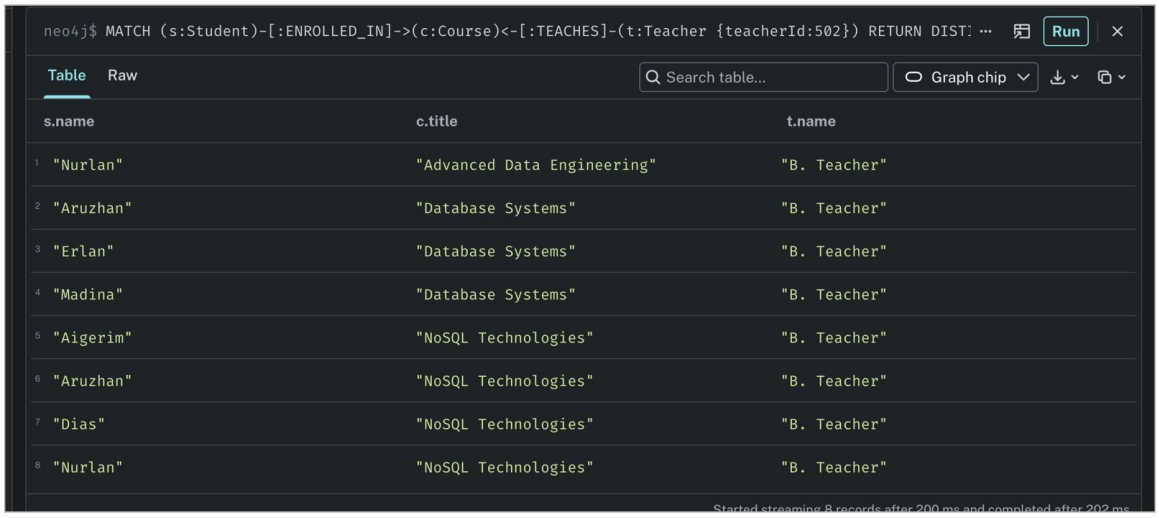


Рисунок 12 - Q3: двухшаговый обход Student - Course - Teacher

Q4. Обход переменной длины: пререквизиты ADV401 до 3 уровней

```
MATCH (c:Course {code:'ADV401'})-[:REQUIRES*1..3]->(p:Course)
RETURN DISTINCT p.code, p.title;
```

Запрос вернул 5 различных пререквизитов: ALG202, NOSQL301, MATH101, PROG101 и DB201 (рисунок 13). Конструкция DISTINCT необходима: до PROG101 можно дойти двумя путями (через ALG202 и через NOSQL301 - DB201), и без DISTINCT он был бы выведен несколько раз. Глубина ограничена числом 3 - это защищает от дорогого неограниченного обхода.

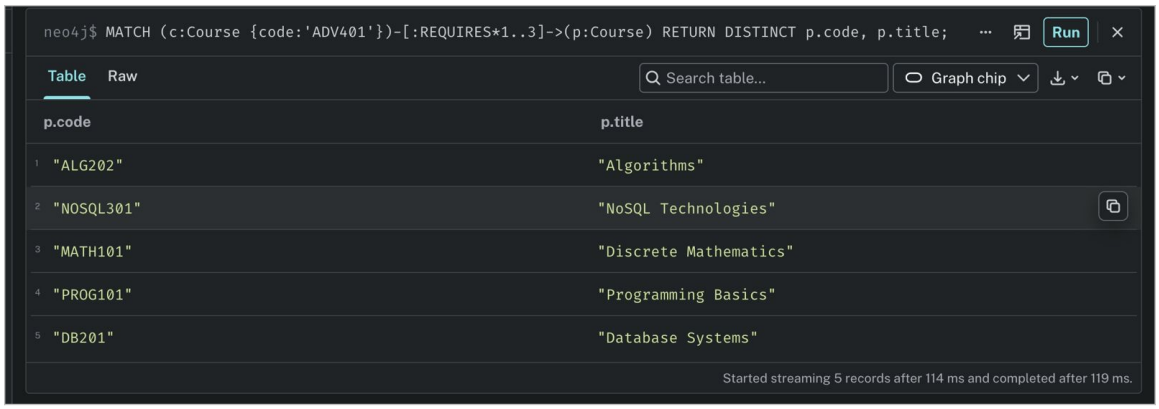


Рисунок 13 - Q4: пререквизиты ADV401 на глубину 1..3

Для повышенного задания варианта выведены сами цепочки и их длина:

```
MATCH path = (c:Course {code:'ADV401'})-[:REQUIRES*1..3]->(p:Course)
RETURN [n IN nodes(path) | n.code] AS chain, length(path) AS depth
ORDER BY depth;
```

Найдено 6 путей: два на глубине 1 (ADV401 - ALG202, ADV401 - NOSQL301), три на глубине 2 и один на глубине 3: ADV401 - NOSQL301 - DB201 - PROG101 (рисунок 14). Самая длинная цепочка пререквизитов ровно равна установленному пределу 3, поэтому ограничение покрывает все пути этого графа. Те же пути в виде подграфа показаны на рисунке 15: 6 узлов и 6 отношений REQUIRES.

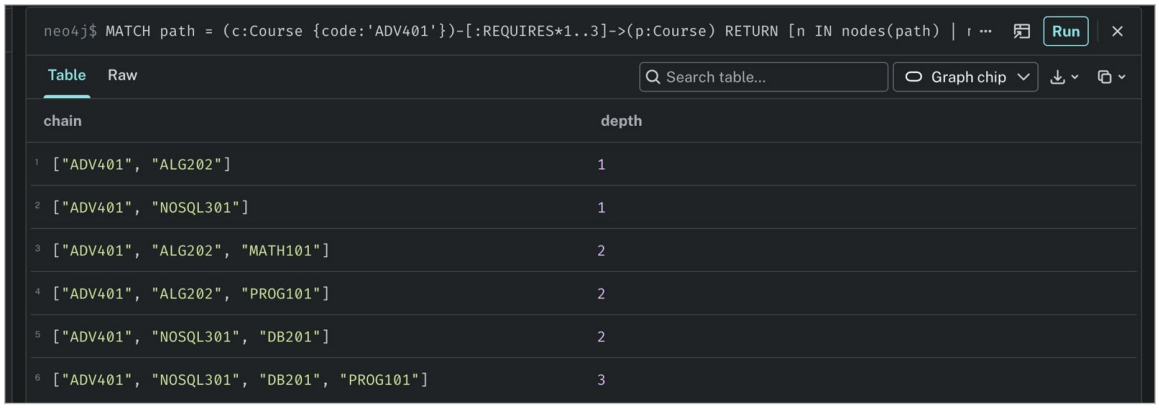


Рисунок 14 - Цепочки пререквизитов ADV401 и их глубина

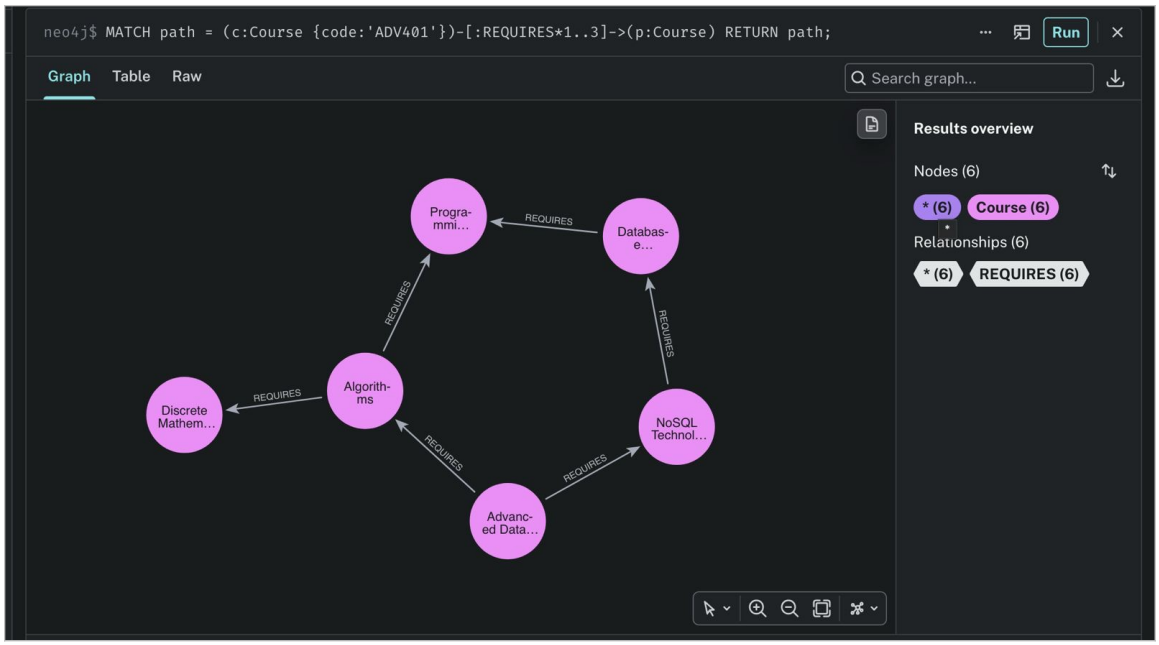


Рисунок 15 - Подграф пререквизитов ADV401 (RETURN path)

Q5. Агрегирование: число студентов на дисциплинах

```
MATCH (c:Course)<-[:ENROLLED_IN]-(s:Student)
RETURN c.code, c.title, count(s) AS students
ORDER BY students DESC;
```

Наиболее востребована NoSQL Technologies (4 студента), далее Database Systems и Programming Basics (по 3), Web Development и Algorithms (по 2), Discrete Mathematics и Advanced Data Engineering (по 1). Сумма 4+3+3+2+2+1+1 = 16 совпадает с числом отношений ENROLLED_IN (рисунок 16).

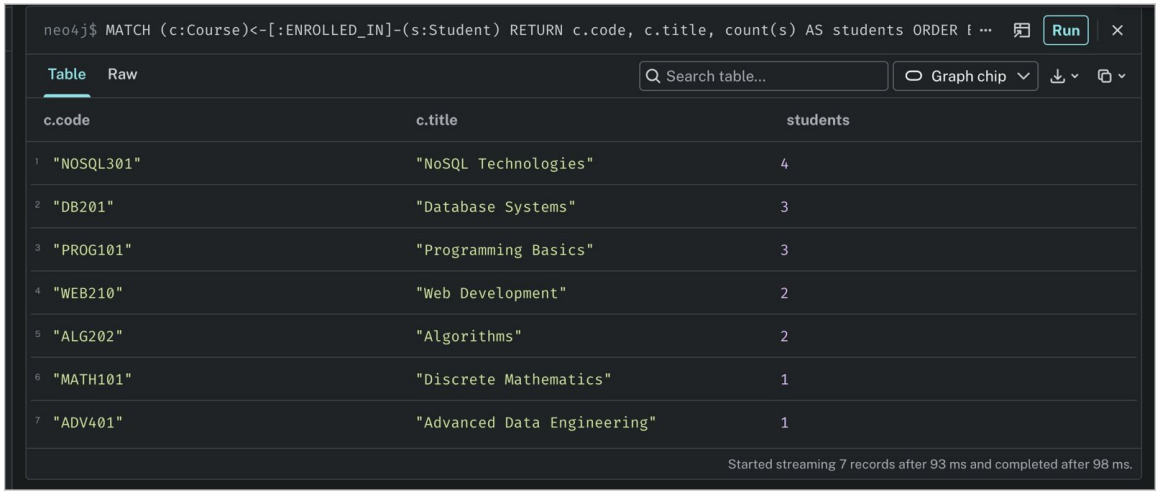


Рисунок 16 - Q5: число студентов по дисциплинам

Q6. Агрегирование по преподавателям

```
MATCH (t:Teacher)-[:TEACHES]->(c:Course)<-[:ENROLLED_IN]-(s:Student)
RETURN t.name, count(DISTINCT c) AS courses,
       count(DISTINCT s) AS students
ORDER BY students DESC;
```

B. Teacher ведёт 3 дисциплины для 6 студентов, A. Teacher - 2 дисциплины для 5 студентов, C. Teacher - 2 дисциплины для 3 студентов (рисунок 17). Использован count(DISTINCT s): студент, посещающий две дисциплины одного преподавателя, не должен считаться дважды. Сумма по строке students (14) больше 8, так как один студент может учиться у разных преподавателей.



Рисунок 17 - Q6: число дисциплин и студентов у преподавателей

Q7. Исключающий шаблон: студенты NOSQL301 без записи на DB201

```
MATCH (s:Student)-[:ENROLLED_IN]->(:Course {code:'NOSQL301'})
WHERE NOT (s)-[:ENROLLED_IN]->(:Course {code:'DB201'})
RETURN s.name;
```

Запрос вернул Nurlan, Aigerim и Dias. Так как NOSQL301 требует DB201, результат показывает студентов, которые сейчас не записаны на пререквизит (рисунок 18). Отношение ENROLLED_IN описывает текущую запись, а не факт сдачи, поэтому для реальной проверки допуска модель нужно расширить статусом «пройдено» (см. раздел 2.9).



Рисунок 18 - Q7: студенты NOSQL301, не записанные на DB201

2.7. Изменение данных (этап 6)

Изменено свойство узла (имя студента 1006) и свойство отношения (статус записи Nurlan на ADV401):

```
MATCH (s:Student {studentId:1006}) SET s.name = 'Nurlan K.' RETURN s;
```

Контрольный запрос показывает узел с новым именем «Nurlan K.» (рисунок 19).

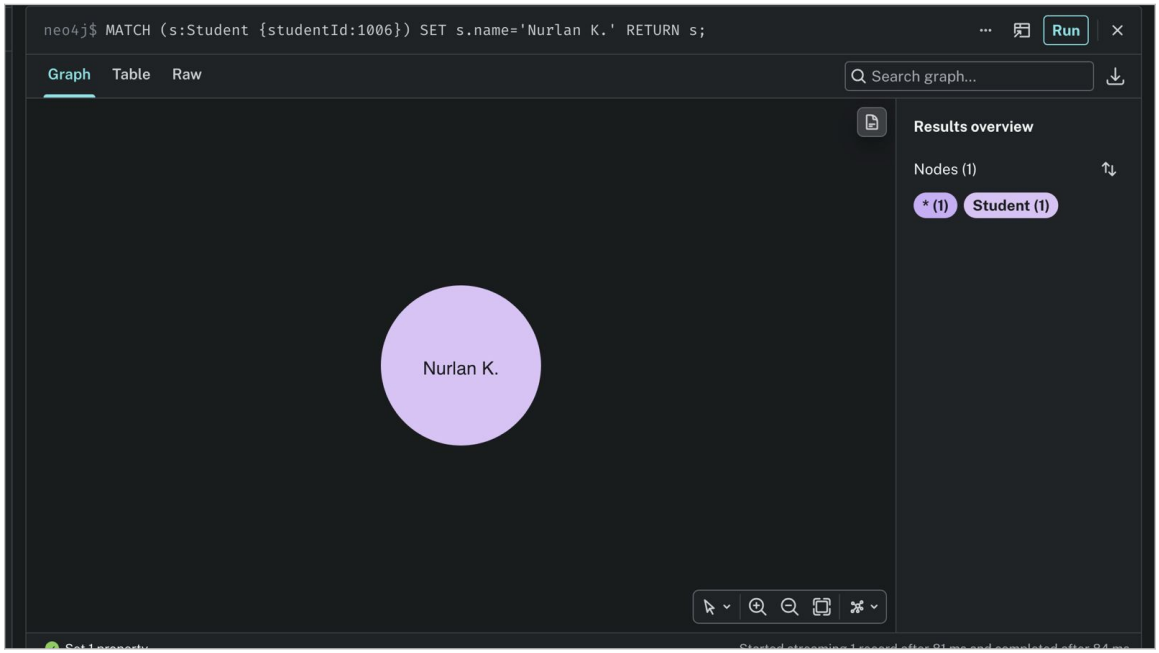


Рисунок 19 - Изменение свойства узла: name = 'Nurlan K.'

```
MATCH (s:Student {studentId:1006})-[r:ENROLLED_IN]->
      (c:Course {code:'ADV401'})
SET r.status = 'confirmed'
RETURN s.name, r, c.code;
```

Запрос изменил одно свойство отношения (Set 1 property) и вернул обновлённое имя студента и связь ENROLLED_IN с ADV401 (рисунок 20). Изменение выполнено после запросов Q1-Q7, поэтому на их рисунках имя ещё записано как Nurlan.



Рисунок 20 - Изменение свойства отношения: status = 'confirmed'

Контрольный запрос по всем записям студента 1006 подтверждает, что изменилась только одна связь: у ADV401 статус «confirmed», у NOSQL301 остался «active» (рисунок 21).

```
MATCH (s:Student {studentId:1006})-[r:ENROLLED_IN]->(c:Course)
RETURN s.name, c.code, r.status;
```



Рисунок 21 - Контрольный запрос: статусы записей студента 1006

2.8. Безопасное удаление (этап 7)

В качестве тестовой связи выбрана запись студента Erlan (1008) на дисциплину PROG101. Сначала элемент показан запросом MATCH ... RETURN, чтобы убедиться, что выбрана ровно одна связь: результат содержит 2 узла и 1 отношение ENROLLED_IN (рисунок 22).

```
MATCH (s:Student {studentId:1008})-[r:ENROLLED_IN]->
      (c:Course {code:'PROG101'})
RETURN s, r, c;
```

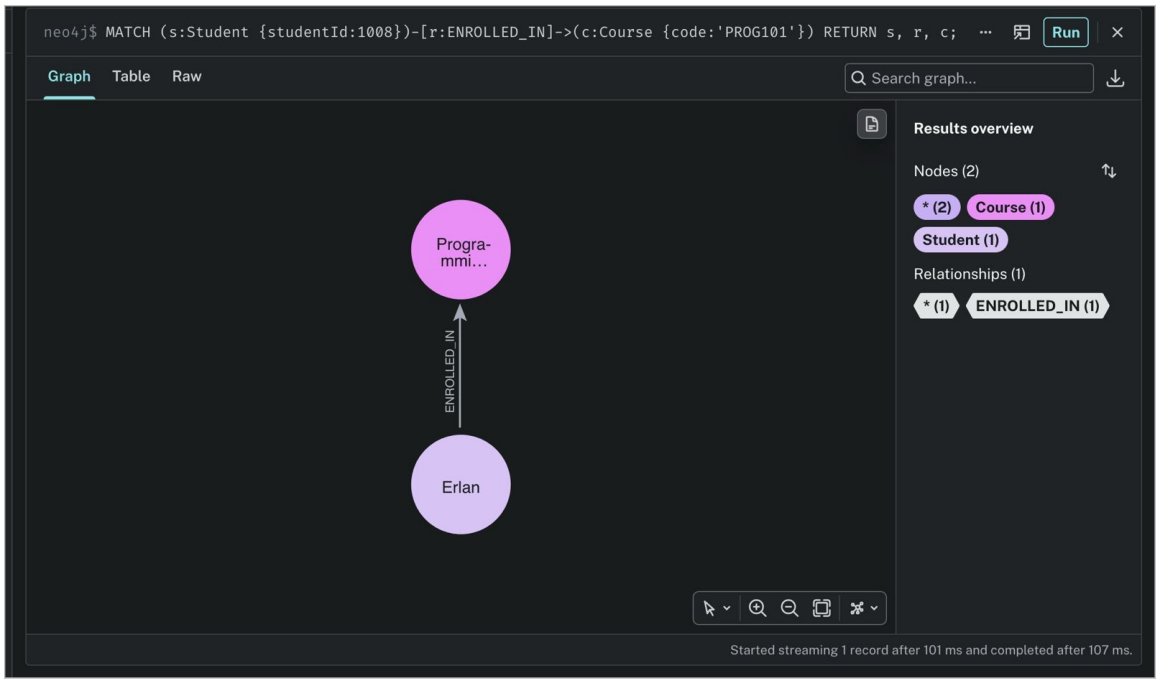


Рисунок 22 - Проверка перед удалением: найдена одна связь Erlan - PROG101

После проверки удалено только отношение r; узлы Student и Course остаются, поэтому DETACH DELETE не требуется:

```
MATCH (s:Student {studentId:1008})-[r:ENROLLED_IN]->
      (c:Course {code:'PROG101'})
DELETE r;
```

Контрольный запрос показал, что число отношений ENROLLED_IN уменьшилось с 16 до 15 (рисунок 23); остальные студенты и дисциплины не затронуты.

```
MATCH ()-[r:ENROLLED_IN]->() RETURN count(r) AS enrollments;
```



Рисунок 23 - Проверка после удаления: ENROLLED_IN - 15

2.9. Повышенный уровень: сравнение моделей и анализ

Для ключевого обхода «пререквизиты дисциплины» сравнены два способа хранения связи: (A) отношение REQUIRES в Neo4j и (B) список идентификаторов prereqIds в документе дисциплины (таблица 7).

Таблица 7 - Связь «дисциплина - пререквизит»: отношение и список идентификаторов

Критерий	**(A) relationship в Neo4j**	**(B) список ID в документе**
Чтение	связь видна в шаблоне запроса, направление задано явно	приходится самому «разворачивать» идентификаторы в документы
Обновление	создать или удалить одно отношение	изменить массив в документе; обратное направление требует отдельного списка
Многошаговый поиск	REQUIRES*1..3 одним запросом (рисунки 13-15)	цикл запросов в приложении, по одному на уровень
Риск рассинхронизации	низкий: связь хранится один раз	высокий при дублировании связи в обоих документах; возможны «висячие» ID

Ограниченный путь 1..3 из ADV401 вернул 6 путей и 5 различных пререквизитов (раздел 2.6, Q4). Максимальная глубина 3 достигнута только по цепочке ADV401 - NOSQL301 - DB201 - PROG101; при добавлении новых дисциплин глубину придётся пересматривать.

Эффективные и сложные запросы

Эффективны запросы, которые стартуют с узла, найденного по уникальному ключу (code, studentId, teacherId), и идут по небольшому числу связей: Q2, Q3, Q4. **Дороже** будут запросы, которые затрагивают все отношения: Q5 и Q6 сканируют все связи ENROLLED_IN, а при росте графа их результат лучше хранить в виде счётчика или считать по периодам. Обход переменной длины при плотных пререквизитах и большой глубине даёт быстрый рост числа путей.

Избыточные связи и ограничения модели

- прямая связь «студент - преподаватель» намеренно не создана, так как выводится через дисциплину;
- узел Semester не введён: семестр хранится в свойстве отношения, поскольку в учебном примере по нему нет обходов; при появлении запросов по семестрам его следует вынести в отдельный узел;
- отношение ENROLLED_IN описывает текущую запись и не хранит факт сдачи, оценки и историю, поэтому проверка допуска по пререквизитам (Q7) пока приблизительна;
- модель не фиксирует, что REQUIRES должен быть ациклическим: цикл пререквизитов запретить средствами схемы нельзя, нужна проверка в приложении.

Сценарий роста графа

Оценка выполнена по допущениям (не измерялась): факультет - около 2 000 студентов и 8 записей на студента за семестр, то есть порядка 16 000 отношений ENROLLED_IN; университет - около 20 000 студентов, то есть порядка 160 000 записей за семестр и около 1,3 млн за 8 семестров истории. Для Neo4j такие объёмы допустимы при наличии ограничений и индексов по ключам поиска. Риски роста: «суперузлы» (дисциплина с тысячами студентов делает дорогим подсчёт и обход через неё), запросы-агрегаты по всему графу и неограниченный обход REQUIRES. Меры: ограничение глубины, ключи уникальности, хранение предвычисленных счётчиков, разделение данных по учебным годам или факультетам на уровне приложения, чтение с реплик в кластере.

Когда документная модель проще: если система только хранит и выдаёт карточки студентов и дисциплин и не строит цепочки связей, документная модель проще в разработке и не требует отдельного языка запросов.

3. Ответы на контрольные вопросы

1. Что такое property graph?

Модель данных, в которой данные представлены узлами, направленными отношениями между ними и свойствами узлов и отношений. Узлы группируются метками, отношения имеют тип.

2. Чем узел отличается от отношения и свойства?

Узел - объект (Student, Course). Отношение - типизированная направленная связь между двумя узлами (ENROLLED_IN). Свойство - атрибут узла или отношения (name, semester).

3. Для чего используются labels в Neo4j?

Метки группируют узлы по ролям, позволяют быстро выбирать узлы нужного вида (MATCH (s:Student)) и задавать ограничения и индексы для конкретной метки.

4. Чем CREATE отличается от MERGE?

CREATE всегда создаёт новый элемент, что при повторном запуске даёт дубли. MERGE ищет шаблон и создаёт его только при отсутствии; вместе с ограничением уникальности делает загрузку повторяемой.

5. Как MATCH описывает графовый шаблон?

Узлы записываются в круглых скобках, отношения - в квадратных, направление - стрелкой, например (s:Student)-[:ENROLLED_IN]->(c:Course). Neo4j находит все подграфы, подходящие под этот шаблон.

6. Почему направление отношения важно?

Направление несёт смысл (студент записан на дисциплину, а не наоборот) и определяет читаемость запросов. Например, REQUIRES идёт от дисциплины к пререквизиту, и обход вдоль стрелки даёт цепочку требований.

7. Когда атрибут следует моделировать свойством, а когда отдельным узлом?

Свойством - если значение описывает сам объект и не участвует в связях (year, credits). Отдельным узлом - если объект участвует в связях и по нему строятся обходы (Course, а при необходимости Semester).

8. Что показывает двухшаговый обход?

Косвенную связь через промежуточный узел. В Q3 обход Student - Course - Teacher показал, у какого преподавателя учатся студенты, хотя прямой связи между ними нет.

9. Почему глубину variable-length path следует ограничивать?

Число путей растёт очень быстро с глубиной; неограниченный обход на большом графе может быть очень дорогим. В работе использован предел 1..3.

10. Почему графовая модель может быть лучше документной для задач анализа связей?

Связи в графе хранятся как самостоятельные элементы, а обходы выполняются одним запросом. В документной модели связи - списки идентификаторов, которые нужно разворачивать вручную и синхронизировать между документами.

4. Вывод

В ходе работы выполнено проектирование и реализация графовой модели Neo4j для варианта 1 «Университет». На основе семи паттернов доступа спроектирована схема из трёх меток (Student, Course, Teacher) и трёх типов направленных отношений (ENROLLED_IN, TEACHES, REQUIRES) с ограничениями уникальности по ключам. Загружено 18 узлов и 30 отношений командами UNWIND + MERGE.

Реализовано 7 запросов Cypher: фильтрация с сортировкой и LIMIT, запрос по отношению, двухшаговый обход, обход переменной длины с ограничением глубины 1..3 (цепочка пререквизитов ADV401 - NOSQL301 - DB201 - PROG101), два агрегирующих запроса count() и исключающий шаблон WHERE NOT. Выполнены изменение свойств узла и отношения с контрольными запросами и безопасное удаление тестовой связи с предварительной проверкой через MATCH ... RETURN.

Для данного кейса граф удобнее документной модели: связи хранятся один раз, цепочки пререквизитов находятся одним запросом, а риск рассинхронизации ниже. Ограничения: модель не хранит историю и факт сдачи дисциплин, а при росте графа дорогими станут агрегаты по всем связям, суперузлы и неограниченные обходы. Их следует сдерживать ограничением глубины, ключами поиска и разделением данных.

5. Список использованных источников

1. Методические указания к лабораторной работе № 7 «Создание графовой модели Neo4j и запросов Cypher» по дисциплине «Проектирование и применение NoSQL-технологий», 2026-2027.
2. Neo4j. Cypher Manual. https://neo4j.com/docs/cypher-manual/current/
3. Neo4j. Graph database concepts (property graph model). https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/
4. Kleppmann, M. Designing Data-Intensive Applications. O'Reilly Media, 2017.